

# Main Memory

---

## Sistem Operasi

Dosen Pengampu: Triawan Adi Cahyanto, M.Kom

Program Studi Teknik Informatika  
Universitas Muhammadiyah Jember  
2025

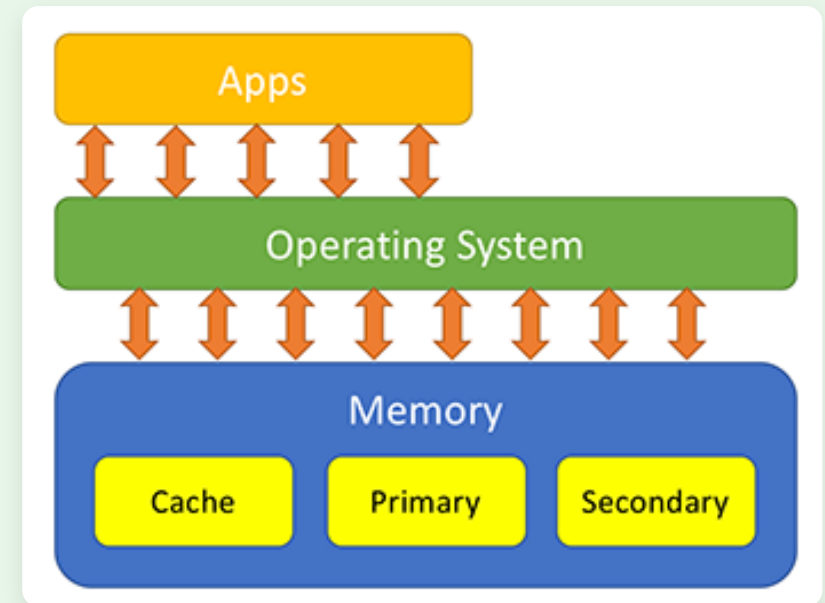


# Background

**Manajemen Memori** adalah fungsi dalam sistem operasi yang mengelola memori utama komputer (RAM) dan memindahkan proses antara memori utama dan disk.

- 📋 **Alokasi & Dealokasi** - Menentukan lokasi memori untuk proses dan membebaskan memori saat tidak digunakan
- 👁️ **Pelacakan Penggunaan** - Memantau bagian memori yang digunakan dan yang tersedia
- 🛡️ **Perlindungan Memori** - Mencegah proses mengakses memori yang tidak diizinkan

Manajemen memori sangat penting untuk menjaga performa sistem, efisiensi penggunaan sumber daya, dan stabilitas sistem operasi.



# Hierarki Memori

## 🔧 Register

Memori tercepat yang terintegrasi dalam CPU

🕒 Kecepatan: Sangat Tinggi    📦 Kapasitas: Sangat Kecil

## ⚡ Cache

Memori kecil yang berfungsi sebagai buffer antara CPU dan RAM

🕒 Kecepatan: Tinggi    📦 Kapasitas: Kecil

## 💻 Main Memory (RAM)

Memori utama yang dapat diakses langsung oleh CPU

🕒 Kecepatan: Sedang    📦 Kapasitas: Sedang

## 📀 Secondary Storage (Disk)

Penyimpanan permanen seperti hard disk atau SSD

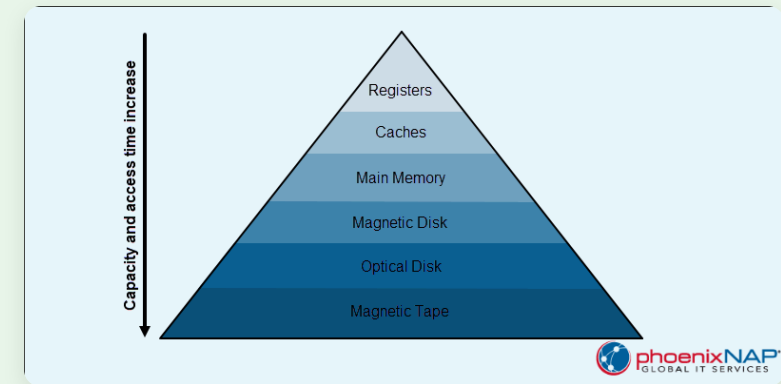
🕒 Kecepatan: Rendah    📦 Kapasitas: Besar

## 📼 Tertiary Storage (Tape)

Penyimpanan backup jangka panjang

🕒 Kecepatan: Sangat Rendah    📦 Kapasitas: Sangat Besar

*Semakin tinggi posisi dalam hierarki, semakin cepat akses namun kapasitas semakin kecil dan biaya semakin mahal*



# Contiguous Memory Allocation

**Alokasi Memori Kontigu** adalah teknik alokasi memori di mana setiap proses ditempatkan dalam satu blok memori yang bersebelahan (tidak terpecah).

## Fixed-Size Partitioning

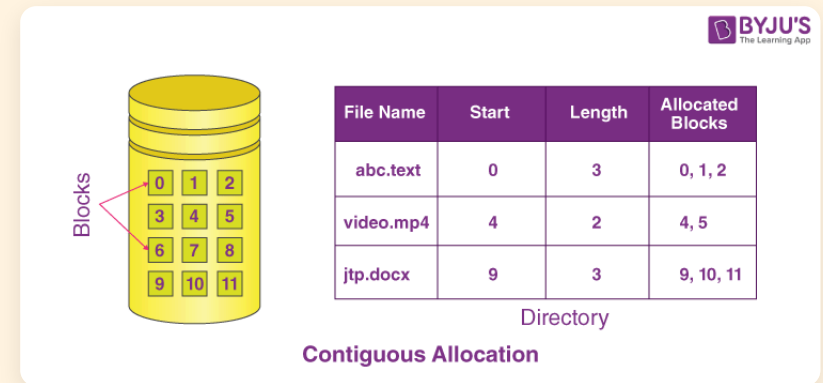
Memori dibagi menjadi partisi dengan ukuran tetap yang sama, setiap proses ditempatkan pada partisi yang cukup besar

## Variable-Size Partitioning

Memori dibagi menjadi partisi dengan ukuran bervariasi sesuai kebutuhan proses, menggunakan algoritma seperti First-Fit, Best-Fit, atau Worst-Fit

### Kelebihan Alokasi Memori Kontigu:

- ✓ Akses memori lebih cepat
- ⚙ Implementasi sederhana
- ⚙ Manajemen memori lebih mudah
- 📈 Performa lebih baik



# Masalah dan Solusi

## ⚠ Fragmentasi

### Fragmentasi Eksternal

Memori yang tersedia tersebar dalam blok-blok kecil yang tidak dapat digunakan untuk proses baru

### Fragmentasi Internal

Memori yang tidak terpakai dalam blok yang dialokasikan untuk proses

## ⚙ Algoritma Penempatan

### → First-Fit

Alokasikan ke partisi pertama yang cukup besar

### ★ Best-Fit

Alokasikan ke partisi terkecil yang cukup besar

### ↗ Worst-Fit

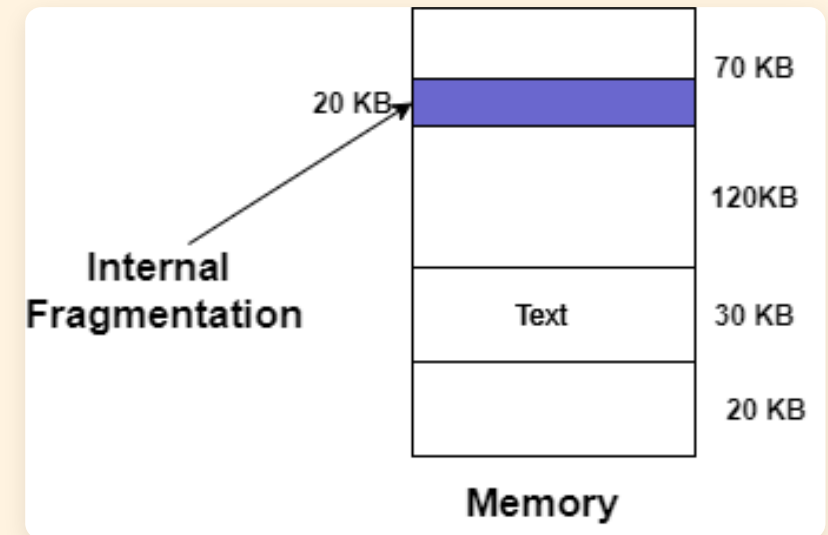
Alokasikan ke partisi terbesar yang tersedia

## 🔧 Solusi

⚡ **Compaction** - Memindahkan semua proses ke satu area untuk menciptakan blok memori yang lebih besar

↔ **Swapping** - Memindahkan proses sementara ke disk untuk membebaskan memori

🗄 **Paging** - Memecah memori fisik menjadi frame dan memori logis menjadi page



# Paging

**Paging** adalah teknik manajemen memori yang memungkinkan proses berada di memori fisik secara **non-kontigu** dengan memecah memori logis dan fisik menjadi blok-blok berukuran sama.

## 📌 Konsep Dasar

### 📄 Page

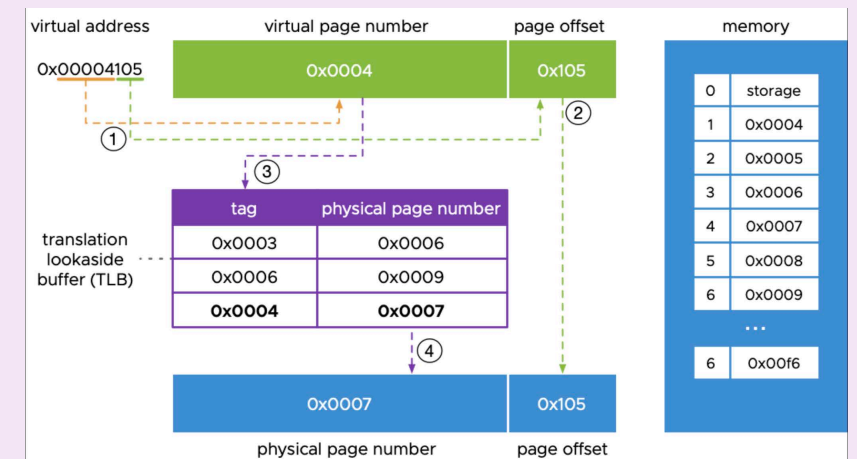
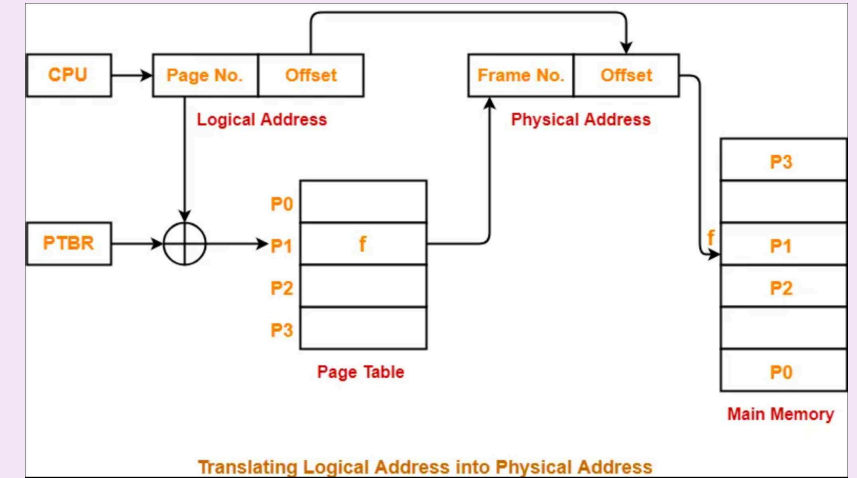
Blok memori logis dengan ukuran tetap dalam ruang alamat proses

### 🏠 Frame

Blok memori fisik dengan ukuran tetap dalam memori utama

## ↔ Cara Kerja Paging

- 1 **Alamat Logis** dibagi menjadi **Page Number** dan **Page Offset**
- 2 **Page Table** memetakan Page Number ke Frame Number
- 3 **Alamat Fisik** = (Frame Number × Frame Size) + Page Offset



# Kelebihan dan Kekurangan

## 👍 Kelebihan Paging

### ✓ Tidak Ada Fragmentasi Eksternal

Memori fisik dapat digunakan secara efisien tanpa adanya ruang kosong yang tidak dapat digunakan

### ✓ Berbagi Memori Efisien

Memungkinkan beberapa proses berbagi kode atau data yang sama tanpa duplikasi

### ✓ Implementasi Virtual Memory

Memungkinkan proses menggunakan lebih banyak memori daripada yang tersedia secara fisik

### ✓ Perlindungan Memori

Setiap proses memiliki ruang alamat sendiri yang terisolasi dari proses lain

## 👎 Kekurangan Paging

### ! Fragmentasi Internal

Bagian terakhir dari setiap page mungkin tidak terisi penuh, menyebabkan pemborosan memori

### ! Overhead Translasi Alamat

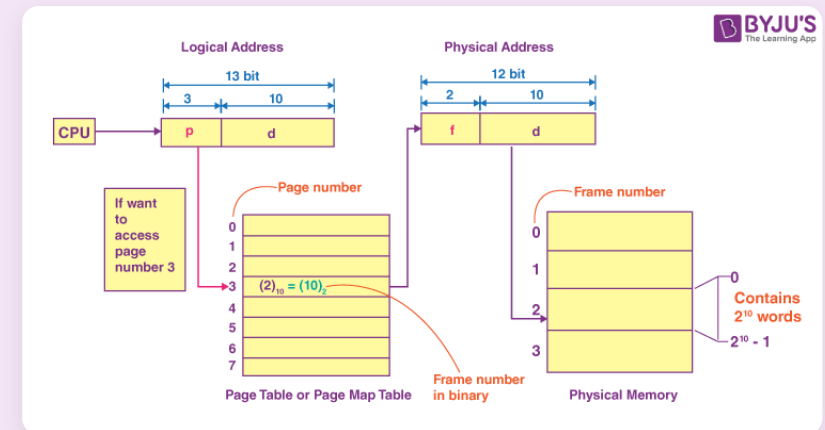
Memerlukan akses tambahan ke page table untuk setiap akses memori

### ! Kompleksitas Implementasi

Memerlukan perangkat keras khusus (MMU) dan struktur data yang kompleks

### ! Page Table Size

Page table dapat menjadi sangat besar untuk proses dengan ruang alamat yang besar



# Structure of the Page Table

**Page Table** adalah struktur data yang menyimpan pemetaan antara alamat virtual (page) dan alamat fisik (frame) dalam sistem operasi.

## Komponen Page Table Entry

### Frame Number

Menunjukkan nomor frame dalam memori fisik

### Valid/Invalid Bit

Menandakan apakah entri valid (1) atau tidak (0)

### Protection Bits

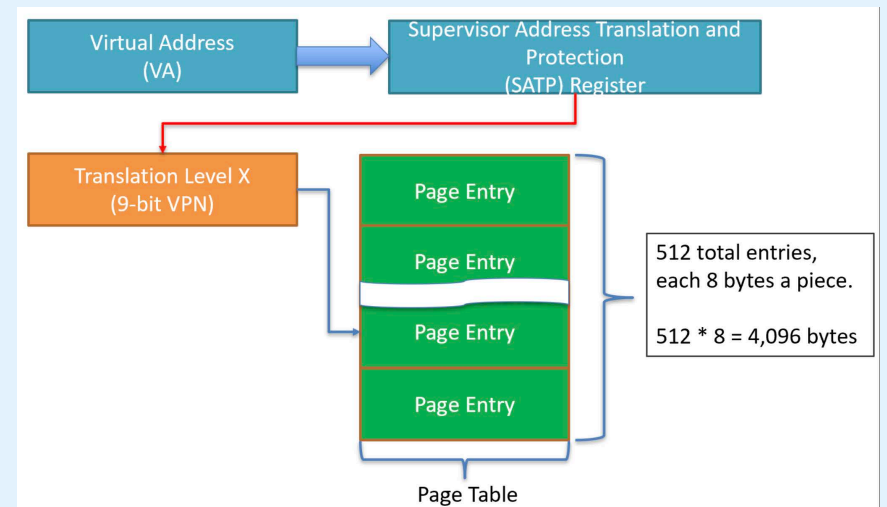
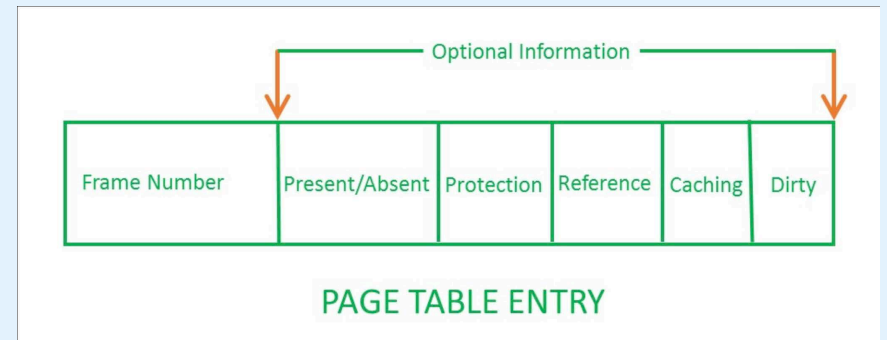
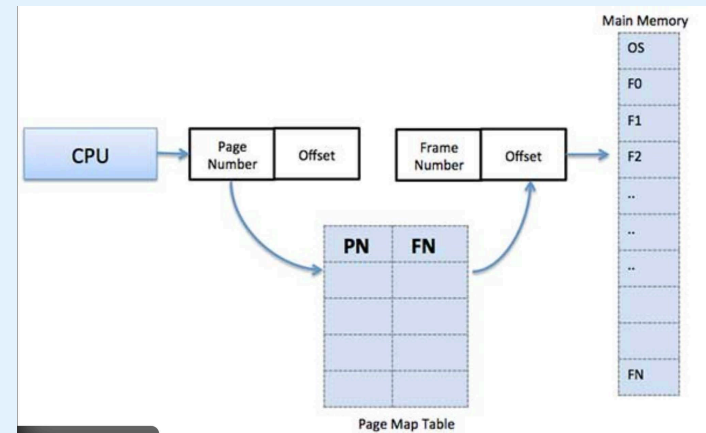
Mengontrol akses (read/write/execute) ke halaman

### Dirty Bit

Menandakan apakah halaman telah dimodifikasi

## Cara Kerja Page Table

- 1 **Alamat Virtual** dibagi menjadi **Page Number** (indeks ke page table) dan **Offset**
- 2 **Page Number** digunakan untuk mengakses entri yang sesuai dalam page table
- 3 **Frame Number** dari entri digabungkan dengan **Offset** untuk membentuk alamat fisik





# Implementasi Page Table yang Efisien

## ⚙️ Teknik Implementasi Page Table

### 🗂️ Hierarchical Paging

Memecah page table menjadi beberapa level untuk mengurangi ukuran dan meningkatkan efisiensi

### # Hashed Page Tables

Menggunakan fungsi hash untuk mengakses entri page table secara efisien

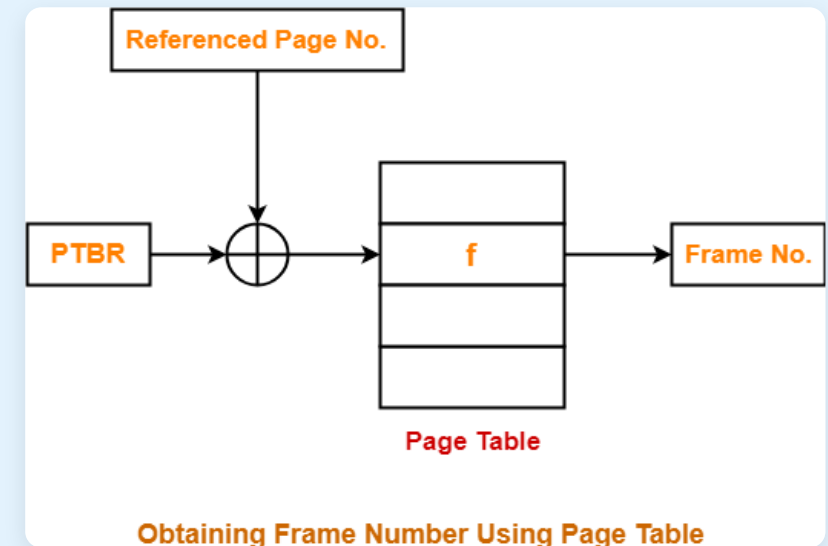
### ↕️ Inverted Page Tables

Satu page table untuk seluruh sistem dengan entri per frame fisik, bukan per proses

## 🔍 Translation Lookaside Buffer (TLB)

**TLB** adalah cache khusus untuk menyimpan pemetaan alamat virtual-fisik yang sering digunakan

- 1 **TLB Hit** - Jika alamat ada di TLB, translasi dilakukan **sangat cepat**
- 2 **TLB Miss** - Jika alamat tidak ada, proses melanjutkan ke page table di memori
- 3 **TLB Update** - Hasil translasi dari page table ditambahkan ke TLB untuk penggunaan berikutnya



# Swapping

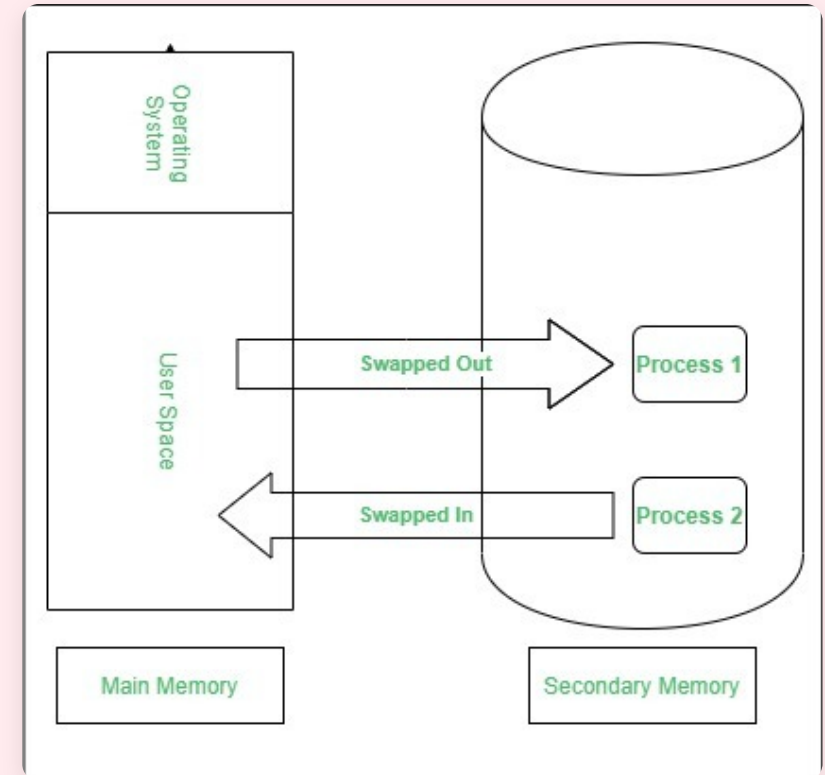
**Swapping** adalah teknik manajemen memori yang memindahkan proses sementara dari memori utama (RAM) ke penyimpanan sekunder (disk) dan sebaliknya untuk mengatasi keterbatasan memori.

## 💡 Tujuan Swapping

- 🔧 **Mengatasi Keterbatasan Memori** - Memungkinkan lebih banyak proses berjalan daripada kapasitas RAM
- 📈 **Optimasi Performa** - Memindahkan proses yang tidak aktif untuk memberikan ruang bagi proses aktif

## ↔ Cara Kerja Swapping

- 1 **Swap Out** - Proses yang tidak aktif atau berprioritas rendah dipindahkan dari RAM ke backing store
- 2 **Swap In** - Proses dikembalikan ke RAM ketika dibutuhkan kembali
- 3 **Context Switch** - Sistem operasi mengelola perpindahan proses secara transparan



# Jenis dan Performa Swapping

## 📌 Jenis Swapping

### ↔ Standard Swapping

Memindahkan **seluruh proses** dari RAM ke disk dan sebaliknya

### 📄 Demand Paging

Memindahkan **hanya halaman yang diperlukan** dari disk ke RAM saat dibutuhkan

## 📌 Faktor Performa

### 📏 Ukuran Swap Space -

Ruang yang dialokasikan untuk swapping



**Kecepatan Disk - SSD**  
lebih cepat daripada HDD



**Algoritma Replacement -**  
Menentukan halaman mana yang akan di-swap

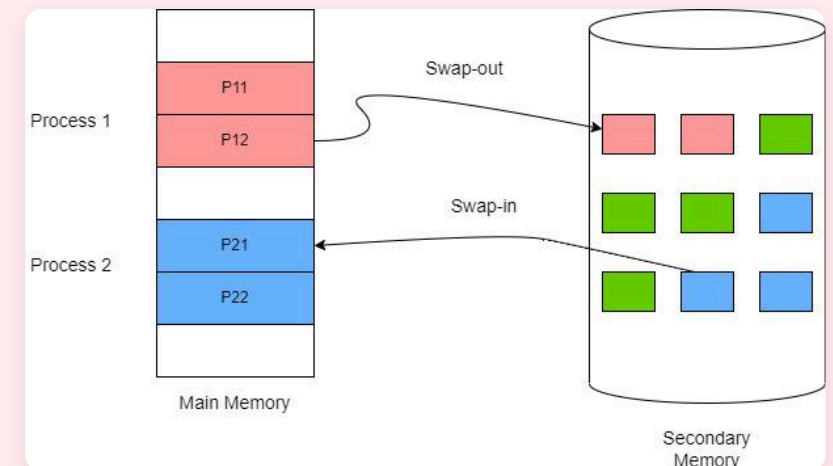
## ↔ Kelebihan dan Kekurangan

### 👍 Kelebihan

- ✓ Memungkinkan lebih banyak proses berjalan
- ✓ Efisien untuk sistem dengan RAM terbatas

### 👎 Kekurangan

- ❗ Performa menurun karena akses disk lebih lambat
- ❗ Risiko thrashing jika terlalu banyak swapping



# Intel 32-bit Architecture

## 👤 Struktur Alamat

### ▮ Segmentation

Membagi ruang alamat logis menjadi **segment** dengan batasan dan perlindungan

### ▤ Paging

Memecah memori fisik menjadi **frame** dengan ukuran 4KB

## ⚙️ Register Manajemen Memori

### ⚙️ CR0

Mengaktifkan mode paging (bit 31)

### ⚠️ CR2

Menyimpan alamat penyebab page fault

### 🔗 CR3

Menyimpan alamat dasar page directory

### ⚙️ CR4

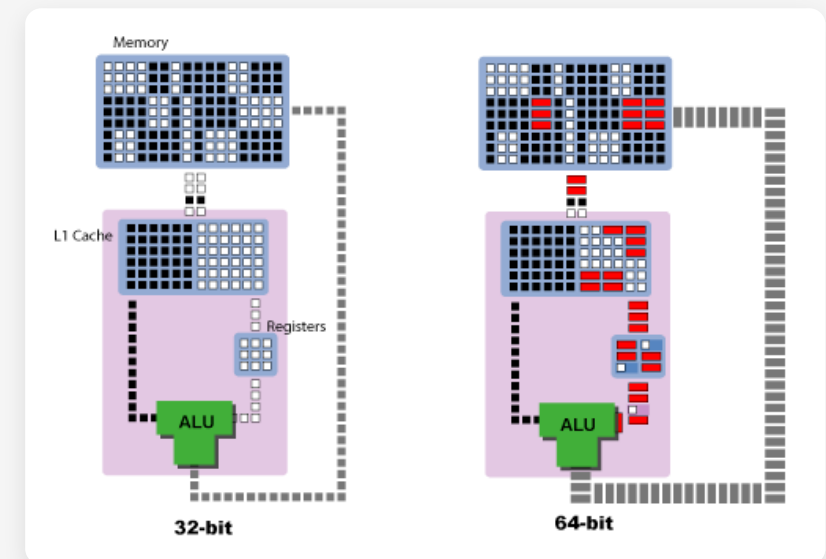
Mengatur fitur paging tambahan

### ▮ Format Page Table

Arsitektur Intel 32-bit menggunakan **two-level page table**:

- Page Directory (**1024 entries**)
- Page Tables (**1024 entries each**)

Setiap entri berisi 32 bit: **20 bit frame number** + **12 bit control**



# Intel 64-bit Architecture

## 📁 Perbedaan Utama 32-bit vs 64-bit

### 🏠 Ruang Alamat

32-bit: 4GB maksimal

64-bit: 16 EB (eksabyte)

### 📝 Register

32-bit: 8 register 32-bit

64-bit: 16 register 64-bit

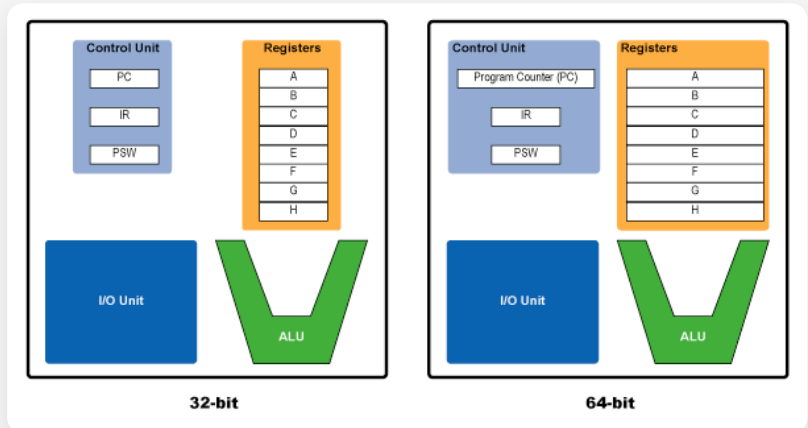
## 🏠 Struktur Alamat 64-bit

### 📁 4-Level Page Table

Intel 64-bit menggunakan hierarki 4 level:

- PML4 (Page Map Level 4)
- PDPT (Page Directory Pointer Table)
- PD (Page Directory)
- PT (Page Table)

Setiap level berisi 512 entries dengan ukuran 8 byte



## 👍 Keuntungan Arsitektur 64-bit

📁 Ruang Alamat Lebih Besar - Mendukung aplikasi yang membutuhkan memori besar

🛡️ Fitur Keamanan - NX bit, SMAP, dan SMEP untuk perlindungan lebih baik

📝 Performa Lebih Baik - Lebih banyak register dan instruksi 64-bit

🔄 Mode Kompatibilitas - Dapat menjalankan aplikasi 32-bit dan 64-bit

# ARMv8 Architecture

**ARMv8** adalah arsitektur prosesor 64-bit yang memperkenalkan eksekusi 64-bit sambil tetap mendukung aplikasi 32-bit.

## Execution States

### AArch32

Mode eksekusi **32-bit** yang kompatibel dengan arsitektur ARMv7

### AArch64

Mode eksekusi **64-bit** dengan register 64-bit dan instruksi baru

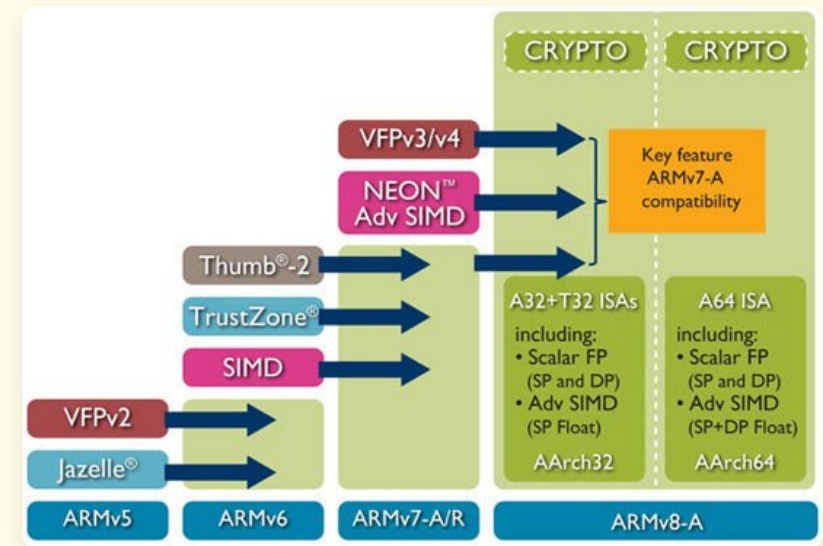
## Manajemen Memori

### Translation Regimes

ARMv8 menggunakan **4-level page table** dengan ukuran entri 8 byte:

- **Level 0:** 1 entry
- **Level 1:** 512 entries
- **Level 2:** 512 entries
- **Level 3:** 512 entries

Mendukung ukuran halaman **4KB**, **16KB**, dan **64KB** dengan granularitas yang berbeda



# Virtualisasi dan Keamanan ARMv8

## Exception Levels

### 1 EL0

Aplikasi  
Mode user  
dengan hak  
akses terbatas

### 2 EL1

OS Kernel  
Mode kernel  
untuk sistem  
operasi

### 3 EL2

Hypervisor  
Mode  
virtualisasi  
untuk  
menjalankan  
VM

### 4 EL3

Secure Monitor  
Mode untuk  
TrustZone dan  
keamanan

## TrustZone

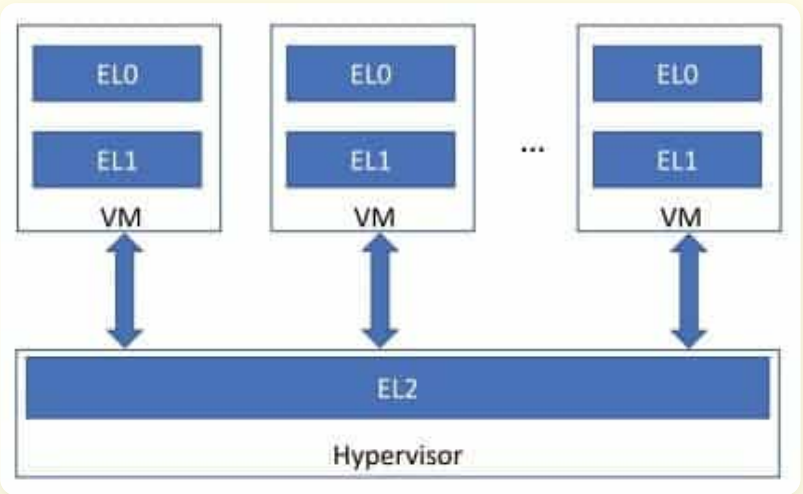
TrustZone adalah teknologi keamanan ARMv8 yang membagi sistem menjadi dua dunia:

- **Normal World** - Untuk aplikasi dan OS biasa
- **Secure World** - Untuk aplikasi dan data sensitif

Monitor mode di EL3 mengontrol transisi antara kedua dunia tersebut

## Perbandingan dengan Intel

- ✓ **ARMv8** - 4 level EL, TrustZone bawaan, lebih efisien untuk perangkat mobile
- ✓ **Intel** - 4 ring privilege, VT-x untuk virtualisasi, lebih dominan di desktop/server



# Summary



## Background

- **Manajemen Memori** - alokasi, dealokasi, pelacakan
- **Hierarki Memori** - register, cache, RAM, disk



## Contiguous Memory

- **Fixed & Variable Partitioning**
- **Fragmentasi** - eksternal & internal
- **Solusi** - compaction, swapping, paging



## Paging

- **Page & Frame** - blok memori berukuran sama
- **Kelebihan** - tidak ada fragmentasi eksternal
- **Kekurangan** - fragmentasi internal, overhead



## Page Table Structure

- **Komponen** - frame number, valid bit, protection
- **Implementasi Efisien** - hierarchical, hashed, inverted
- **TLB** - cache untuk translasi alamat



## Swapping

- **Standard vs Demand Paging**
- **Faktor Performa** - ukuran swap, kecepatan disk
- **Kelebihan/Kekurangan** - efisiensi vs performa



## Arsitektur

- **Intel** - 32-bit (2-level) vs 64-bit (4-level)
- **ARMv8** - AArch32/AArch64, 4-level page table
- **Virtualisasi & Keamanan** - EL0-EL3, TrustZone



# Terima Kasih

